

Effect of operator<=> on the C++ Standard Library

Document Number: P0790R1

Date: 2018-08-06

Author: David Stone (davidmstone@google.com, david@doublewise.net)

Audience: LEWG, LWG

This paper lists (what are expected to be) non-controversial changes to the C++ standard library in response to [P0515](#), which adds operator<=> to the language. This is expected to be non-controversial because it tries to match existing behavior as much as possible. As a result, all proposed additions are either strong_equality or strong_ordering, matching the existing comparison operators.

This document should contain a complete list of types or categories of types in C++.

Revision History

R1: A much broader version of this paper was presented to LEWG at a previous meeting. What remains in this paper is everything which the group did not find controversial and which probably does not require significant justification. All controversial aspects will be submitted in separate papers.

Backward Compatibility

The operator<=> proposal was written such that the "generated" operators are equivalent to source code rewrites – there is no actual operator== that a user could take the address of. Users are not allowed to form pointers to standard library member functions and are unable to form pointers to friend functions defined inline in the class. There are some cases where we do not specify how the operator was implemented, only that the expression a @ b is valid; these cases are not broken by such a change because users could not have depended on it, anyway. In general, we accept changes that overload existing functions, which also has the effect of breaking code which takes the address of a free function.

Types that are not proposed to get operator<=> in this paper

These types are not comparable now. This paper does not propose adding any new comparisons to any of these types.

- deprecated types
- exception types
- tag classes (nothrow, piecewise_construct_t, etc.)
- arithmetic function objects (plus, minus, etc.)
- comparison function objects (equal_to, etc.)
- owner_less
- logical function objects (logical_and, etc.)
- bitwise function objects (bit_and, etc.)
- nested_exception
- allocator_traits
- char_traits
- iterator_traits
- numeric_limits
- pointer_traits
- regex_traits
- chrono::duration_values

- tuple_element
- max_align_t
- map::node_type
- map::insert_return_type
- set::node_type
- set::insert_return_type
- unordered_map::node_type
- unordered_map::insert_return_type
- unordered_set::node_type
- unordered_set::insert_return_type
- any
- default_delete
- aligned_storage
- aligned_union
- system_clock
- steady_clock
- high_resolution_clock
- locale::facet
- locale::id
- ctype_base
- ctype
- ctype_byname
- codecvt_base
- codecvt
- codecvt_byname
- num_get
- num_put
- numpunct
- numpunct_byname
- collate
- collate_byname
- time_get
- time_get_byname
- time_put
- time_put_byname
- money_base
- money_get
- money_put
- money_punct
- moneypunct_byname
- message_base
- messages
- messages_byname
- FILE
- va_list
- back_insert_iterator
- front_insert_iterator
- insert_iterator
- ostream_iterator
- ostreambuf_iterator
- ios_base
- ios_base::Init
- basic_ios
- basic_streambuf

- `basic_istream`
- `basic_iostream`
- `basic_ostream`
- `basic_stringbuf`
- `basic_istreamstream`
- `basic_ostringstream`
- `basic_stringstream`
- `basic_filebuf`
- `basic_ifstream`
- `basic_ofstream`
- `basic_fstream`
- `gslice`
- `slice_array`
- `gslice_array`
- `mask_array`
- `indirect_array`
- `atomic_flag`
- `thread`
- `mutex`
- `recursive_mutex`
- `timed_mutex`
- `timed_recursive_mutex`
- `lock_guard`
- `scoped_lock`
- `unique_lock`
- `once_flag`
- `shared_mutex`
- `shared_timed_mutex`
- `shared_lock`
- `condition_variable`
- `condition_variable_any`
- `promise`
- `future`
- `shared_future`
- `packaged_task`
- `random_device`
- `hash`
- `weak_ptr`
- `basic_regex`
- `sequential_execution_policy`
- `parallel_execution_policy`
- `parallel_vector_execution_policy`
- `default_searcher`
- `boyer_moore_searcher`
- `boyer_moore_horspool_searcher`
- `ratio`
- `integer_sequence`
- `seed_seq` (paper needed to add `strong_equality`)
- `enable_shared_from_this`: It would be nice to give it a `strong_ordering` to allow derived classes to = default. However, this means that all classes that do not explicitly delete their comparison operator get an `operator<=>` that compares only the `enable_shared_from_this` base class, which is almost certainly wrong. Since this is intended to be used as a base class, we should not add `operator<=>` to it. Moreover, classes which `enable_shared_from_this` are unlikely to be basic value classes so they do not lose much by not being able to default.

- `initializer_list`: `initializer_list` is a reference type. It would be strange to give it reference semantics on copy but value semantics for comparison. It would also be surprising if two `initializer_list` containing the same set of values compared as not equal. Therefore, I recommend not defining it for this type.

Types from C that are not proposed to get operator<=> in this paper

- `div_t`
- `ldiv_t`
- `lldiv_t`
- `imaxdiv_t`
- `timespec`
- `tm`
- `lconv`
- `fenv_t`
- `fpos_t`
- `mbstate_t`

Types that should get operator<=>, no change from current comparisons

These types are all currently comparable.

- `error_category`: `strong_ordering`
- `error_code`: `strong_ordering`
- `error_condition`: `strong_ordering`
- `exception_ptr`: `strong_ordering`
- `type_info`: `strong_equality`
- `monostate`: `strong_ordering`
- `bitset`: `strong_equality` (paper would be needed to change this to `strong_ordering`)
- `allocator`: `strong_equality`
- `memory_resource`: `strong_equality`
- `synchronized_pool_resource`: (implicitly from `memory_resource` base class)
- `unsynchronized_pool_resource`: (implicitly from `memory_resource` base class)
- `monotonic_buffer_resource`: (implicitly from `memory_resource` base class)
- `polymorphic_allocator`: `strong_equality`
- `scoped_allocator_adaptor`: `strong_equality`
- `pool_options`: `strong_equality`
- `function`: `strong_equality` with `nullptr_t` only (no homogenous operator)
- `chrono::duration`: `strong_ordering`, heterogeneous with durations of other representations and periods
- `chrono::time_point`: `strong_ordering`, heterogeneous in the duration
- `type_index`: `strong_ordering`
- `locale`: `strong_equality`
- `complex`: `strong_equality` (heterogeneous with `T` and homogeneous)
- `linear_congruential_engine`: `strong_equality`
- `mersenne_twister_engine`: `strong_equality`
- `subtract_with_carry_engine`: `strong_equality`
- `discard_block_engine`: `strong_equality`
- `independent_bits_engine`: `strong_equality`
- `shuffle_order_engine`: `strong_equality`
- `uniform_int_distribution`: `strong_equality`
- `uniform_int_distribution::param_type`: `strong_equality`
- `uniform_real_distribution`: `strong_equality`

- `uniform_real_distribution::param_type`: `strong_equality`
- `bernoulli_distribution`: `strong_equality`
- `bernoulli_distribution::param_type`: `strong_equality`
- `binomial_distribution`: `strong_equality`
- `binomial_distribution::param_type`: `strong_equality`
- `geometric_distribution`: `strong_equality`
- `geometric_distribution::param_type`: `strong_equality`
- `negative_binomial_distribution`: `strong_equality`
- `negative_binomial_distribution::param_type`: `strong_equality`
- `poisson_distribution`: `strong_equality`
- `poisson_distribution::param_type`: `strong_equality`
- `exponential_distribution`: `strong_equality`
- `exponential_distribution::param_type`: `strong_equality`
- `gamma_distribution`: `strong_equality`
- `gamma_distribution::param_type`: `strong_equality`
- `weibull_distribution`: `strong_equality`
- `weibull_distribution::param_type`: `strong_equality`
- `extreme_value_distribution`: `strong_equality`
- `extreme_value_distribution::param_type`: `strong_equality`
- `normal_distribution`: `strong_equality`
- `normal_distribution::param_type`: `strong_equality`
- `lognormal_distribution`: `strong_equality`
- `lognormal_distribution::param_type`: `strong_equality`
- `chi_squared_distribution`: `strong_equality`
- `chi_squared_distribution::param_type`: `strong_equality`
- `cauchy_distribution`: `strong_equality`
- `cauchy_distribution::param_type`: `strong_equality`
- `fisher_f_distribution`: `strong_equality`
- `fisher_f_distribution::param_type`: `strong_equality`
- `student_t_distribution`: `strong_equality`
- `student_t_distribution::param_type`: `strong_equality`
- `discrete_distribution`: `strong_equality`
- `discrete_distribution::param_type`: `strong_equality`
- `piecewise_constant_distribution`: `strong_equality`
- `piecewise_constant_distribution::param_type`: `strong_equality`
- `piecewise_linear_distribution`: `strong_equality`
- `piecewise_linear_distribution::param_type`: `strong_equality`
- `filesystem::path`: `strong_ordering`
- `filesystem::path::iterator`: `strong_ordering`
- `filesystem::directory_entry`: `strong_ordering`
- `filesystem::directory_iterator`: `strong_ordering`
- `filesystem::recursive_directory_iterator`: `strong_ordering`
- `istream_iterator`: `strong_equality`
- `istreambuf_iterator`: `strong_equality`
- `match_results`: `strong_equality`
- `regex_iterator`: `strong_equality`
- `regex_token_iterator`: `strong_equality`
- `thread::id`: `strong_ordering`
- `fpos`: `strong_equality`
- `array::iterator`: `strong_ordering`
- `deque::iterator`: `strong_ordering`
- `forward_list::iterator`: `strong_equality`
- `list::iterator`: `strong_equality`
- `vector::iterator`: `strong_ordering`

- `map::iterator`: `strong_equality`
- `set::iterator`: `strong_equality`
- `multimap::iterator`: `strong_equality`
- `multiset::iterator`: `strong_equality`
- `unordered_map::iterator`: `strong_equality`
- `unordered_set::iterator`: `strong_equality`
- `unordered_multimap::iterator`: `strong_equality`
- `unordered_multiset::iterator`: `strong_equality`
- `valarray::iterator`: `strong_ordering`

Types that will get their operator<=> from a conversion operator

These types will get operator<=> if possible without any changes, just like they already have whatever comparison operators their underlying type has.

- `integral_constant` and all types deriving from `integral_constant` (has operator `T`)
- `bitset::reference` (has operator `bool`)
- `reference_wrapper` (has operator `T &`)
- `atomic` (has operator `T`)

This has the disadvantage that types which have a template comparison operator will not have their wrapper convertible. For instance, `std::reference_wrapper<std::string>` is not currently comparable. This does not affect `bitset::reference`, as it has a fixed conversion to `bool`, but it does affect the other three.

Types that wrap another type

- `array`
- `deque`
- `forward_list`
- `list`
- `vector` (including `vector<bool>`)
- `map`
- `set`
- `multimap`
- `multiset`
- `unordered_map`
- `unordered_set`
- `unordered_multimap`
- `unordered_multiset`
- `queue`
- `queue::iterator`
- `priority_queue`
- `priority_queue::iterator`
- `stack`
- `stack::iterator`
- `pair`
- `tuple`
- `reverse_iterator`
- `move_iterator`
- `optional`
- `variant`

This turned out to be much more complicated than expected and will require its own paper.

basic_string, basic_string_view, char_traits, and sub_match

Properly integrating `operator<=>` with these types requires more thought than this paper has room for, and thus will be discussed separately.

unique_ptr and shared_ptr

They contain state that is not observed in the comparison operators. Therefore, they will get their own paper.

valarray

Current comparison operators return a `valarray<bool>`, giving you the result for each pair (with undefined behavior for differently-sized `valarray` arguments). It might make sense to provide some sort of function that returns `valarray<comparison_category>`, but that should not be named `operator<=>`. This paper does not suggest adding `operator<=>` to `valarray`.

Types that have no comparisons now but are being proposed to get operator<=> in another paper

This paper does not propose changing any of the following types -- they are here only for completeness.

- `filesystem::file_status`
- `filesystem::space_info`
- `slice`
- `to_chars_result`
- `from_chars_result`

nullptr_t

Already supports `strong_equality` in the working draft. I will be writing a separate paper proposing `strong_ordering`.

Not Updating Concepts That Provide Comparisons

This category includes things like `BinaryPredicate` and `Compare`. This is addressed in a separate paper.

Not Updating Concepts That Require Comparisons

This includes things like `LessThanComparable` and `EqualityComparable`. This is addressed in a separate paper.

Miscellaneous

All `operator<=>` should be `constexpr` and `noexcept` where possible, following the lead of the language feature and allowing `= default` as an implementation strategy for some types.

When we list a result type as "unspecified" it is unspecified whether it has `operator<=>`. There are not any unspecified result types for which we currently guarantee any comparison operators are present, so there is no extra work to do here.